# Lab 05 — Questions

### Question 1 [Analysis]

A team delivers its Spring Boot API in a 950 MB image built from `maven:3.9-eclipse-temurin-21`. The security officer refuses to put it into production. Give **three** arguments that have nothing to do with disk space, then say which one is the hardest to fix other than with a multi-stage build.

### Question 2 [Understanding]

In a multi-stage Dockerfile with three `FROM`s, which one produces the final image? What becomes of the others? And if no `COPY --from` references the second stage, what does Buildah do — and how do you see it in the output of `podman build`?

### Question 3 [Diagnosis]

A developer writes:

```
FROM docker.io/library/maven:3.9-eclipse-temurin-21 AS build
COPY . /app
WORKDIR /app
RUN mvn package -DskipTests

FROM docker.io/library/eclipse-temurin:21-jre-alpine
COPY --from=build /app /app
ENTRYPOINT ["java","-jar","/app/target/api.jar"]
```

The final image is 420 MB instead of the expected 210 MB, and the source code is still in it. Explain the mistake and fix it in one line.

### Question 4 [Analysis]

Your colleague wants to "save time" by containerising `ng serve`: the image contains Node, the project sources, and starts the Angular development server on port 4200. It works in staging. Give four reasons to reject this image in production, then describe in two sentences what to do instead.

### Question 5 [Analysis]

A team migrates its image from `eclipse-temurin:21-jre` (Ubuntu) to `eclipse-temurin:21-jre-alpine` and saves 120 MB. Two weeks later, a PDF generation job fails in production with an `UnsatisfiedLinkError`. Explain the probable link, say which command in each image would have shown the difference, and how this migration should have been conducted.

### Question 6 [Understanding]

Why is multi-stage said to be the only **reliable** protection against build secrets, when you can also delete the file with an `rm`? In which case does multi-stage **not** protect?

### Question 7 [Analysis]

Compare these two strategies for a 50 MB Spring Boot JAR, from the point of view of the **deployment time** of a one-line fix: (a) `COPY target/api.jar app.jar`, (b) layered extraction (`-Djarmode=tools … extract --layers`). Quantify approximately what is transferred in each case, and say why (a) remains acceptable in many companies.

### Question 8 [Diagnosis]

A CI build goes from 90 seconds to 7 minutes after moving to a new agent, without any file having changed. The Dockerfile is correctly ordered (dependencies before code). Explain, and give two mechanisms that bring the 90 seconds back — stating, for an agent that builds with rootless Podman, where the cache lives.

### Question 9 [Analysis]

A *distroless* image strongly reduces the attack surface. Name two operational capabilities you concretely lose, and say how a team usually compensates for each.

### Question 10 [Understanding]

`RUN --mount=type=cache,target=/root/.m2 mvn package`: where is that data stored, and why does it not appear in the final image? How is it different from a `VOLUME`? And what happens if you forget the `# syntax=docker/dockerfile:1` line — under Docker, then under Podman?

### Question 11 [Analysis]

A developer claims: "Multi-stage is useless for Angular, since the result is only static files anyway." Answer them by describing what the image would contain without multi-stage, and the size gap at stake.

### Question 12 [Diagnosis]

The following Dockerfile fails with `COPY failed: … /app/dist: no such file or directory`:

```
FROM docker.io/library/node:22-alpine AS build
WORKDIR /src
COPY . .
RUN npm ci && npm run build

FROM docker.io/library/nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
```

Find the error, then say which `podman build` command would let you inspect the real content of the `build` stage to diagnose it without guessing.

### Question 13 [Analysis]

Your company requires unit tests to fail the image build. Where do you put `RUN mvn test` in a multi-stage Dockerfile, and what is the drawback of this approach compared with tests run upstream by CI?

### Question 14 [Analysis]

Two images of the same application: one of 250 MB in a single layer, the other of 280 MB in five layers of which four are stable. Which one deploys faster on an update of the application code? Justify, and say in which situation the answer reverses.